

SE446: Big Data Engineering

Week 8A: Spark on the Cluster — Deployment & Performance Tuning

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Instructor Notes

Welcome to Week 8A. Today we move from running Spark interactively on the shell to deploying real jobs on a YARN cluster. This is a critical transition because everything you have done so far — RDDs, DataFrames, Spark SQL — was executed locally or in an interactive session. As such, you never had to worry about cluster resources, memory allocation, or network shuffles. Today we change that. We will learn how to submit jobs with `spark-submit`, how to read the Spark Web UI to diagnose problems, and how to apply three performance levers: caching, partition tuning, and broadcast joins. These are the skills you will need for your M3 milestone.

Today's Agenda

Instructor Notes

Let us look at the agenda for today. We have five main sections. First, a brief recap of where we are in the course journey. Second, we will learn **spark-submit** and the two deploy modes. Third, we will do a deep dive into the Spark Web UI so you can read execution plans and spot bottlenecks. Fourth, we will cover caching and persistence to avoid redundant HDFS reads. Fifth, we will discuss partition tuning and broadcast joins. It is important to note that each of these topics builds on the previous one, so pay close attention to the sequence.

Learning Objectives

By the end of this session you will be able to:

- 1 Submit Spark jobs to a YARN cluster using `spark-submit`
- 2 Distinguish `client` vs `cluster` deploy modes
- 3 Read the Spark Web UI: Jobs, Stages, Tasks, Storage, Executors
- 4 Apply caching with `.cache()` and `.persist()`
- 5 Tune partition count for optimal parallelism
- 6 Use broadcast joins for small lookup tables

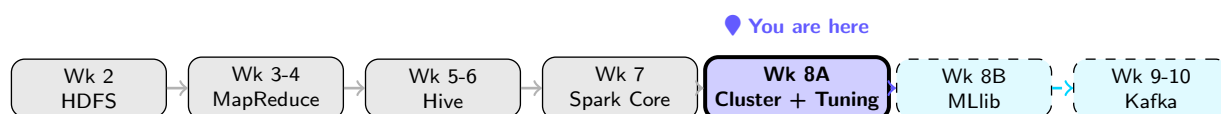
Dataset

Chicago Crimes (HDFS) — same dataset from Week 7

Instructor Notes

These six learning objectives represent the core competencies you need to move from Spark prototyping to Spark production. The first two are about deployment — getting your code onto the cluster. The third is about observability — understanding what your job is actually doing. The last three are about performance tuning. It is important to note that we are using the same Chicago Crimes dataset from Week 7, so you already know the schema and the data. This lets us focus entirely on the infrastructure and tuning aspects without getting distracted by data exploration.

The Big Data Journey — Where Are We?



✓ What you already know

- RDDs, DataFrames, Spark SQL
- PySpark in **interactive shell**
- Ran jobs locally or on the shell

🚀 What changes today

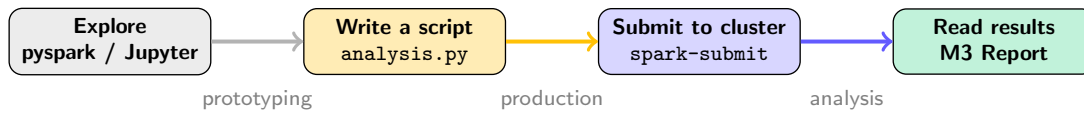
- Submit **real jobs** to the cluster
- Read the **Spark Web UI**
- Make jobs **fast** (tuning)
- **M3 milestone** uses these skills

Instructor Notes

Let us take a moment to see where we are in the course. You have already learned HDFS for storage, MapReduce for distributed computation, Hive for SQL on Hadoop, and last week you learned Spark Core with RDDs and DataFrames. On the left side, everything you have done so far was interactive — you typed commands in the PySpark shell and saw results immediately. On the right side, today we shift to production mode. You will submit batch jobs, read execution metrics in the Web UI, and apply tuning techniques. This is why this week matters for M3 — your milestone requires running a complete analytics pipeline on the cluster, not just typing commands interactively.

Outline

From Exploration to Submission — Your M3 Workflow



> PySpark Shell / Jupyter

- Type code, see results instantly
- Great for **exploring** the dataset
- Driver runs on **your machine**
- Cannot run unattended

🔧 spark-submit on YARN

- Runs a `.py` script as a batch job
- Works on the **cluster's resources**
- Reproducible — versioned in GitHub
- Required for **M3 submission**

The Rule

Prototype in Jupyter → polish to `.py` → `spark-submit` → submit results for M3.

Instructor Notes

This slide shows the workflow you will follow for every Spark project, including M3. The first step is exploration — you open PySpark or Jupyter, load the data, try transformations, and see what works. The second step is to take your working code and put it into a standalone Python script. The third step is to submit that script to the YARN cluster using `spark-submit`. It is important to note that the PySpark shell is great for prototyping, but it cannot run unattended and it ties up your terminal. With `spark-submit`, your job runs on the cluster's resources, it is reproducible, and you can version it in GitHub. This is why M3 requires `spark-submit` — we want you to practice the production workflow.

spark-submit Syntax

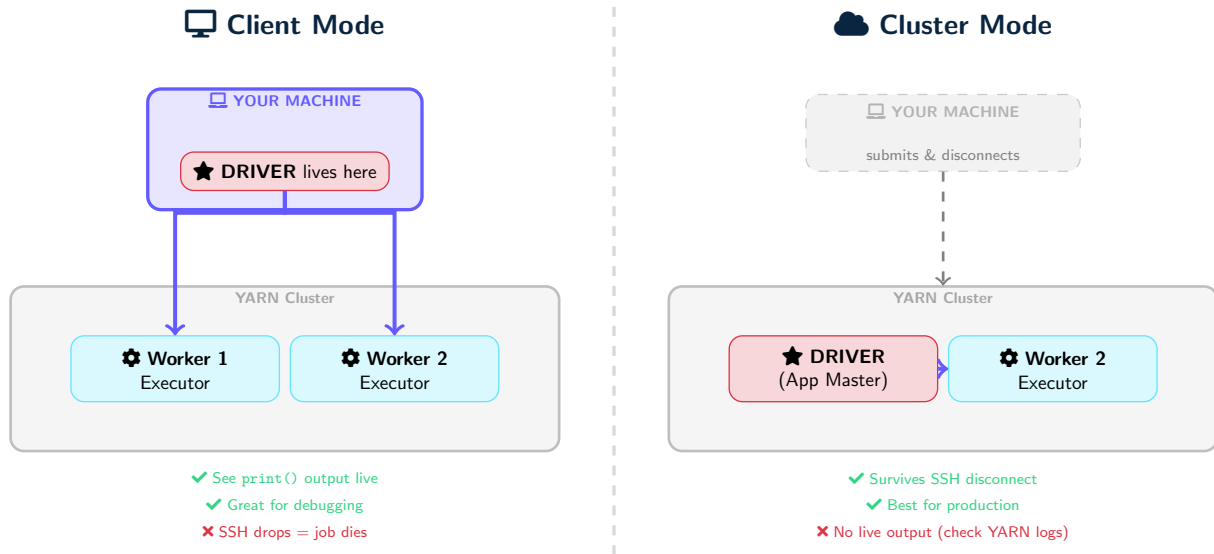
```
spark-submit \  
  --master yarn \  
  --deploy-mode client \  
  --num-executors 2 \  
  --executor-memory 1g \  
  --executor-cores 2 \  
  --driver-memory 512m \  
  --conf spark.sql.shuffle.partitions=4 \  
  crime_analysis.py
```

Flag	Purpose
--master yarn	Run on YARN cluster
--deploy-mode	Where Driver runs (client/cluster)
--num-executors	Number of executor JVMs
--executor-memory	RAM per executor
--executor-cores	CPU cores per executor
--conf	Custom Spark configuration

Instructor Notes

Let us walk through each flag in this command because you will be typing this many times. The `--master yarn` flag tells Spark to use YARN as the resource manager instead of running locally. The `--deploy-mode` decides where the driver lives, which we will cover in detail on the next slide. The `--num-executors 2` means we want two executor processes — one on each worker node in our cluster. Now here is the important one: `--executor-memory 1g` sets one gigabyte of RAM per executor. Why one gigabyte and not two? Because each worker node only has two gigabytes total, and YARN itself needs some overhead memory. If you request two gigabytes per executor, YARN will reject your job. The `--executor-cores 2` gives each executor two CPU cores for parallel task execution. Finally, the `--conf` flag lets you set any Spark configuration property, and we will see later why `shuffle.partitions` is critical.

Deploy Mode: client vs cluster



Instructor Notes

This is one of the most important diagrams in today's lecture. To understand deploy modes, we need to answer one question: where does the Driver process live? In client mode, on the left, the Driver runs on your machine — meaning your laptop or your SSH session to the master node. As a consequence, you see all `print()` output live in your terminal, which is excellent for debugging. However, if your SSH connection drops, the Driver dies and the entire job fails. In cluster mode, on the right, your machine simply submits the job and disconnects. The Driver runs inside YARN as the Application Master. As such, the job survives even if you close your laptop. The tradeoff is that you cannot see live output — you must check YARN logs afterward. We will see how to retrieve those logs shortly.

Deploy Modes — Think of It Like a Phone Call

Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (print() output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**

Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**

Common Pitfall

Students often use cluster mode during development and wonder why they see no output. **Use client mode first**, switch to cluster for final submission.

Quick Check: You're debugging and need to see print() output. Which mode?

Deploy Modes — Think of It Like a Phone Call

Client Mode = Speakerphone

- You're on the call the **entire time**
- You hear everything live (print() output)
- If you **hang up** (SSH drops), the call ends
- Best for: **development & debugging**

Cluster Mode = Voicemail

- You leave a message and **hang up**
- The work happens **without you**
- Check results later via YARN logs
- Best for: **production & long jobs**

Common Pitfall

Students often use cluster mode during development and wonder why they see no output. **Use client mode first**, switch to cluster for final submission.

Quick Check: You're debugging and need to see print() output. Which mode?

Client mode!

Instructor Notes

Let us understand this through a simple analogy. Client mode is like being on a speakerphone call — you are present the entire time, you hear everything live, but if you hang up the call is over. Cluster mode is like leaving a voicemail — you leave the instructions and walk away, and the work happens without you. The common pitfall I see every semester is students using cluster mode during development and then wondering why they see no output in their terminal. This is why the rule is simple: use client mode while you are developing and debugging, then switch to cluster mode only for your final production runs or long-running jobs.

Setting Up Spark Client on Your Computer

1. Install Spark & Set Environment

Download the **same Spark version** as your cluster from spark.apache.org, then:

```
export SPARK_HOME=/opt/spark           # where you extracted Spark
export PATH=$SPARK_HOME/bin:$PATH
export HADOOP_CONF_DIR=/etc/hadoop/conf
```

2. Point to Your Cluster

Copy these three files from the cluster into \$HADOOP_CONF_DIR: `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`. Your machine must be able to reach the cluster nodes over the network.

3. Submit in Client Mode

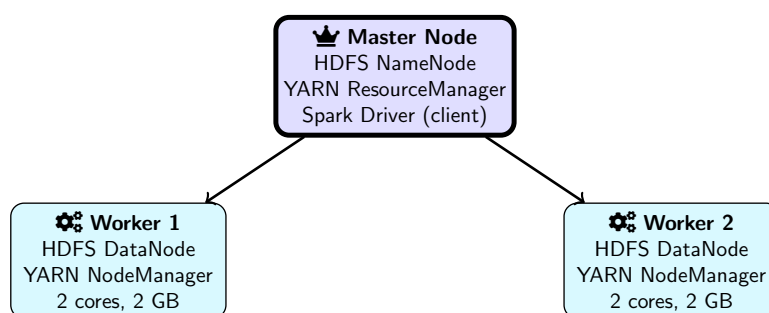
```
spark-submit --master yarn --deploy-mode client \
  --num-executors 2 --executor-memory 1g my_analysis.py
```

Your laptop is the **Driver** → `print()` output appears in your terminal.

Instructor Notes

Now let me show you how to actually set this up on your own laptop so you can work in client mode. There are three steps, and none of them is complicated once you know what to do. First, you download Apache Spark — and this is important — the same version that is running on your cluster. If the cluster runs Spark 3.5, you download 3.5. A version mismatch will give you cryptic serialization errors that are painful to debug. You extract it, set `SPARK_HOME`, add the `bin` directory to your `PATH`, and set `HADOOP_CONF_DIR` to point to where the Hadoop config files will live. Second, you need to tell Spark where your cluster is. You do this by copying three XML files from the cluster: `core-site.xml`, `hdfs-site.xml`, and `yarn-site.xml`. These tell Spark where HDFS is, where YARN is, and how to talk to them. Place them in the directory you just pointed `HADOOP_CONF_DIR` to. Make sure your machine can actually reach the cluster over the network — firewalls and VPNs are the usual culprits when this fails. Third, you run `spark-submit` with `--deploy-mode client`. This means your laptop becomes the Driver. The executors still run on the cluster, but the Driver sits on your machine, so every `print()` statement shows up right in your terminal. That is incredibly useful for development and debugging. The catch, of course, is that your machine must stay connected. If your SSH drops or your laptop goes to sleep, the Driver dies and the whole job fails. For long unattended jobs, switch to cluster mode.

Our Cluster Configuration



Spark Web UI: <http://master:4040> — YARN UI: <http://master:8088>

Cluster Resources

Total: 4 cores, 4 GB RAM
Request $\leq$ 1g per executor

Analogy

Master = **project manager** (coordinates)
Workers = **team members** (do the work)

Instructor Notes

This is the actual cluster you will be working on in the lab. The master node runs three services: the HDFS NameNode that tracks where data blocks are stored, the YARN ResourceManager that allocates containers to applications, and in client mode, your Spark Driver. Each worker node has two cores and two gigabytes of RAM, and runs the HDFS DataNode and YARN NodeManager. The total cluster gives us four cores and four gigabytes. It is important to note that you should request at most one gigabyte per executor, because YARN needs overhead memory for its own processes. Think of the master as the project manager who coordinates, and the workers as the team members who do the actual computation.

Understanding the Cluster — Resource Math

Master = Coordinator

- **NameNode**: knows where data blocks are
- **ResourceManager**: allocates containers
- **Driver** (client): plans execution

Workers = Muscle

- **DataNode**: stores data blocks
- **NodeManager**: manages containers
- **Executors**: run your tasks

Resource Math

Total cluster: 4 cores, 4 GB. With `--num-executors 2 --executor-cores 2 --executor-memory 1g`, you use **all 4 cores** and **2 GB** (leaving headroom for OS and YARN overhead).

Instructor Notes

Let us do the resource math together because this is where students often make mistakes. We have two workers, each with two cores and two gigabytes. When we request two executors with two cores and one gigabyte each, we are using all four cores but only two gigabytes out of four. Why not request two gigabytes per executor? Because the operating system and YARN NodeManager on each worker also need memory. If you over-allocate, YARN will either reject your job or the executor will be killed with an out-of-memory error. As a consequence, the safe formula for our cluster is: two executors, two cores each, one gigabyte each. This leaves enough headroom for the system processes to run comfortably.

Example Script: Crime Counts

```
# crime_counts.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

spark = SparkSession.builder \
    .appName("CrimeCounts") \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

result = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")) \
    .orderBy(col("cnt").desc())

result.show(10)
spark.stop()
```

Submit to YARN

```
spark-submit --master yarn --deploy-mode client crime_counts.py
```

Instructor Notes

This is a complete, self-contained PySpark script that you could submit to the cluster right now. Notice the structure: first we create a `SparkSession`, then we read data from HDFS, apply transformations, call an action, and finally call `spark.stop()`. It is important to note that `spark.stop()` at the end is not optional — without it, your application holds onto cluster resources even after it finishes. The submit command at the bottom is the simplest form: just `--master yarn` and `--deploy-mode client`. We are not specifying executor memory or cores here, so Spark will use defaults. In the lab, you will add those flags explicitly to control resource allocation.

Think-Pair-Share: spark-submit

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to `spark-submit`?

Discuss with your neighbor for 30 seconds.

You have a script that takes 30 minutes to run.

Your laptop might lose WiFi during the run.

Which deploy mode should you use?

What flag do you add to spark-submit?

Discuss with your neighbor for 30 seconds.

✓ Answer

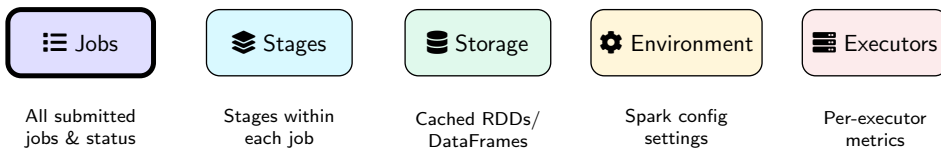
`--deploy-mode cluster` — the Driver runs inside YARN, so even if your WiFi drops, the job continues. Retrieve output with `yarn logs -applicationId <id>`.

Instructor Notes

This is a quick exercise to check your understanding. Take thirty seconds and discuss with your neighbor. The scenario is clear: a long-running job and an unreliable connection. If you use client mode and your WiFi drops, the Driver dies and thirty minutes of computation are lost. As such, the answer is cluster mode. The flag is `--deploy-mode cluster`. To retrieve the output afterward, you use the `yarn logs` command with the application ID that YARN assigned to your job. This is exactly the kind of decision you will face when submitting your M3 analysis.

Outline

Spark Web UI — Overview



While App Runs

`http://master:4040`

After App Finishes

YARN proxy:

`http://master:8088/proxy/<app-id>/`

Key Principle

Each **action** creates a **job**. Each job has one or more **stages** (separated by shuffles). Each stage has multiple **tasks** (one per partition).

Instructor Notes

The Spark Web UI is your window into what your job is actually doing on the cluster. There are five tabs, and each one answers a different question. The Jobs tab shows you all submitted jobs and their status. The Stages tab lets you drill into the stages within each job. The Storage tab shows which DataFrames are cached in memory. The Environment tab shows your Spark configuration. The Executors tab shows per-executor metrics like memory usage and garbage collection time. While your application is running, you access it at port 4040 on the master node. After the application finishes, you use the YARN proxy URL. It is important to note the hierarchy: one action creates one job, shuffles split a job into stages, and each stage has one task per partition.

Predict the Execution Plan

```
df.groupBy("Primary Type").count().show()
```

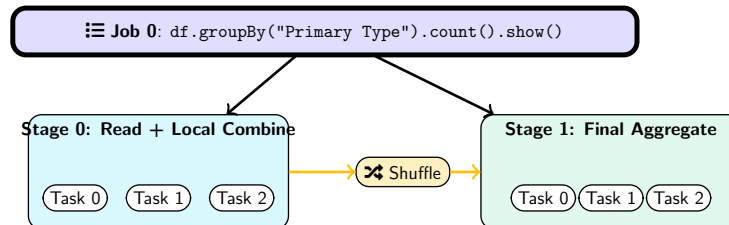
Before I show you the answer: How many **jobs**? How many **stages**? Why?
Hint: Which operation forces a shuffle?

Predict the Execution Plan

```
df.groupBy("Primary Type").count().show()
```

Before I show you the answer: How many **jobs**? How many **stages**? Why?

Hint: Which operation forces a shuffle?



✓ Answer

1 job (one action: `.show()`), **2 stages** (separated by the `groupBy` shuffle).

Instructor Notes

Before I reveal the answer, think about it. How many jobs? There is exactly one action here — `.show()` — so there is exactly one job. How many stages? The `groupBy` operation requires a shuffle because rows with the same primary type need to be co-located on the same partition for counting. As a consequence, the job is split into two stages. Stage zero reads the data from HDFS and performs a local partial aggregation on each partition. Then the shuffle moves data so that all rows with the same key land on the same partition. Stage one performs the final aggregation. Each stage has multiple tasks — one per partition. This is exactly what you will see in the Spark Web UI when you run this in the lab.

Understanding Jobs, Stages, and Tasks

Jobs — Triggered by Actions

- Every **action** (`.show()`, `.count()`, `.collect()`, `.write`) creates a new **Job**
- Transformations (`.filter()`, `.groupBy()`) are lazy — they do **not** trigger Jobs

Stages — Separated by Shuffles

- A Job is split into **Stages** at every **shuffle boundary** (`groupBy`, `join`)
- Fewer stages = fewer shuffles = **better performance**

Tasks — One Per Partition

- Each stage launches **one task per partition** (4 partitions = 4 tasks)
- Tasks run in **parallel** across executor cores
- A slow task (straggler) slows the **entire stage** — check the Tasks tab!

Instructor Notes

Let us formalize the hierarchy. The first level is jobs — every action in your code creates a new job. Transformations like `filter` and `groupBy` are lazy and do not trigger jobs by themselves. The second level is stages — a job is split into stages at every shuffle boundary. Operations like `groupBy` and `join` require shuffles because they need data to be reorganized across partitions. Fewer stages means fewer shuffles, which means better performance. The third level is tasks — each stage launches one task per partition, and these tasks run in parallel across your executor cores. It is important to note that a single slow task, called a straggler, will hold up the entire stage. You can spot stragglers in the Tasks tab of the Web UI.

Spotting Performance Problems in the Web UI

Symptom	Where to Look	Cause & Fix
One task 10× slower	Tasks tab — Duration	Data skew: one partition too large. Fix: <code>repartition()</code> or salting.
High GC Time	Executors tab	Memory pressure: increase <code>executor-memory</code> or reduce partition size.
Large shuffle spill	Stages tab — Spill column	Not enough RAM: data spills to disk. Fix: more memory or fewer partitions.
Many empty tasks	Stages tab — Task count	Over-partitioned: too many partitions. Fix: <code>coalesce()</code> .

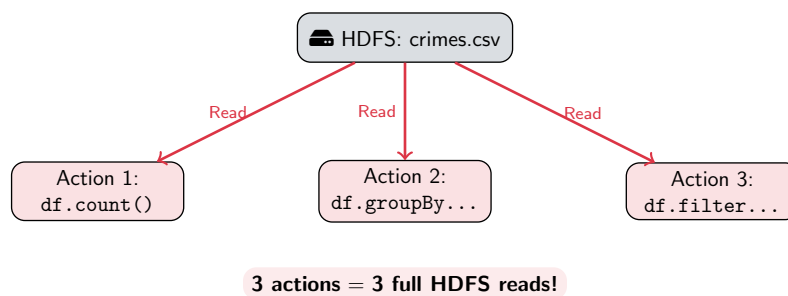
Lab Task

In today's lab, you'll observe each of these in the real Spark Web UI while running experiments on the cluster.

Instructor Notes

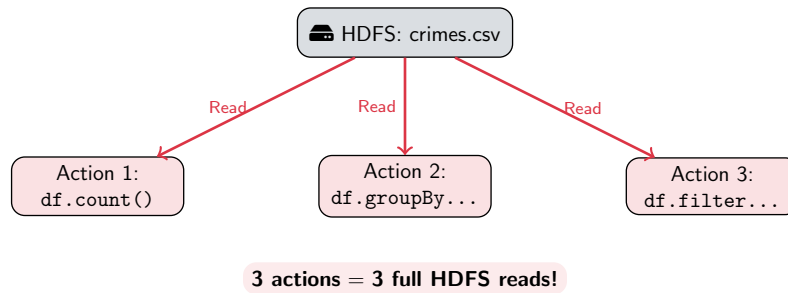
This table is your diagnostic reference guide. When something is slow, the Web UI tells you why — but you need to know where to look. The first symptom is one task taking ten times longer than others, which means data skew — one partition has far more data. The fix is to `repartition()` or use salting. The second symptom is high garbage collection time on the Executors tab, which means memory pressure. The third is shuffle spill on the Stages tab, which means data is spilling from memory to disk during a shuffle. The fourth is many empty tasks, which means you have too many partitions for your data size. In today's lab, you will deliberately create each of these problems and observe them in the Web UI, so you learn to diagnose them.

The Problem: Repeated HDFS Reads



Quiz: You run 5 actions without caching. How many HDFS reads?

The Problem: Repeated HDFS Reads



Quiz: You run 5 actions without caching. How many HDFS reads? **5 times!**

Why?

Spark is **lazy** — every action recomputes from scratch by replaying the DAG lineage.

Instructor Notes

Imagine that you have a DataFrame loaded from HDFS and you need to run three different analyses on it: a count, a group-by, and a filter. Without caching, Spark reads the entire CSV file from HDFS three separate times — once for each action. Why? Because Spark is lazy. It does not store intermediate results by default. Every time you call an action, Spark replays the entire DAG lineage from the very beginning, starting with reading the file from disk. With ten thousand rows this is tolerable, but with millions of rows each read can take minutes. As a consequence, three actions means three full HDFS reads, and five actions means five full reads. This is the problem that caching solves.

Why Does Spark Re-Read? — The Library Analogy

✗ Without Caching

- Like walking to the shelf **every time** you need a fact
- Need the population of Riyadh? Walk to shelf.
- Need it again? Walk **again**.
- 5 questions = 5 trips

✓ With Caching

- Like **photocopying** the page
- First trip: walk + photocopy
- Every question after: read from **your desk**
- 5 questions = 1 trip + 4 instant reads

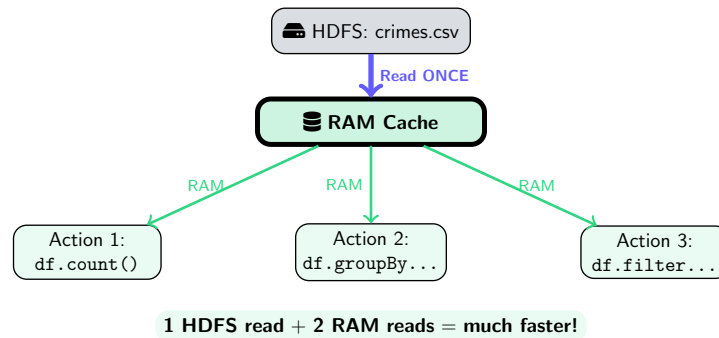
📈 The Cost in Practice

- `df.count()` — reads 10,000 rows from HDFS
- `df.groupBy(...).show()` — reads the **same** 10,000 rows again
- With real datasets (millions of rows), each read can take **minutes**

Instructor Notes

Let us understand this through a simple analogy. Imagine you are in a library doing research. Without caching, every time you need a fact you walk back to the shelf, pull out the book, find the page, read the fact, and walk back. If you need the same fact again five minutes later, you walk all the way back again. This is exactly what Spark does without caching — it goes back to HDFS every single time. With caching, it is like photocopying the page on your first trip. You walk once, make a copy, and keep it on your desk. Every subsequent question is answered instantly from your desk copy. In practice, the first action still takes the full time, but every action after that reads from RAM, which is ten to one hundred times faster than HDFS.

The Solution: `.cache()` and `.persist()`



How It Works

- 1 `df.cache()` **marks** for caching (lazy!)
- 2 First action: reads HDFS, stores in RAM
- 3 Next actions: from **RAM** — 10–100× faster

Lab 07 Benchmark

Without cache: 21s + 21s + 21s = **63s**

With cache: 21s + 0.8s + 3.3s = **25s**

Speedup: **2.5×**

Instructor Notes

Now let us see the solution. With caching, the data flows from HDFS into RAM once, and all subsequent actions read from RAM. The three-step process is important to understand. First, `df.cache()` marks the DataFrame for caching — but it does not actually cache anything yet because Spark is lazy. Second, the first action triggers the actual read from HDFS and stores the result in executor memory. Third, every subsequent action reads from RAM instead of HDFS. Look at the benchmark numbers from Lab 07: without caching, three actions took sixty-three seconds total. With caching, the same three actions took twenty-five seconds — a 2.5 times speedup. The first action still takes twenty-one seconds, but the second and third are nearly instant.

Caching in Code

```
# .cache() = persist in RAM only (default)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)
df.cache()           # marks for caching (lazy!)
df.count()           # 1st action: reads HDFS, stores in RAM
df.groupBy("Primary Type").count().show() # from RAM!

# Unpersist when done (free memory)
df.unpersist()
```

Common Mistake

`df.cache()` does **NOT** cache immediately!
Caching happens on the **first action**.

Memory Tip

If DataFrame > RAM, Spark evicts partitions. Use
`.persist(MEMORY_AND_DISK)` instead.

Instructor Notes

Let us walk through this code line by line. We read the CSV from HDFS, then call `df.cache()` which only marks the DataFrame for caching — nothing is stored in memory yet. The first action, `df.count()`, triggers the actual HDFS read and stores the data in RAM. The second action, `groupBy`, reads from RAM and is much faster. At the end, we call `df.unpersist()` to free the memory. The most common mistake students make is assuming that `df.cache()` immediately loads data into memory. It does not — it is lazy, just like transformations. If your DataFrame is larger than available RAM, Spark will silently evict partitions, and those partitions will be re-read from HDFS on the next action. We will see how to handle that case on the next slide.

Storage Levels — `.persist()`

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)  # spill to disk
df.persist(StorageLevel.DISK_ONLY)        # disk only
```

Level	RAM	Disk	Repl.	Use Case
MEMORY_ONLY	✓	✗	No	Default (<code>.cache()</code>). Fastest.
MEMORY_AND_DISK	✓	✓	No	Large data that may not fit
DISK_ONLY	✗	✓	No	Very large data, save RAM
MEMORY_ONLY_2	✓	✗	2×	Critical data, fault tolerance

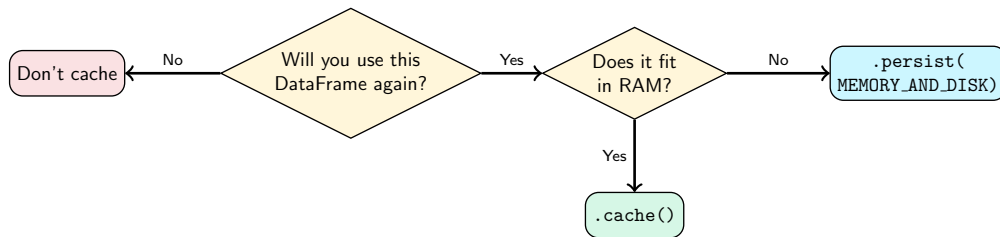
Rule of Thumb

Start with `.cache()` (`MEMORY_ONLY`). If you get `OutOfMemory`, switch to `MEMORY_AND_DISK`.

Instructor Notes

While `.cache()` is convenient, it only stores data in RAM. The `.persist()` method gives you fine-grained control over where data is stored. The first level, `MEMORY_ONLY`, is what `.cache()` uses by default — it is the fastest but if data does not fit, partitions are evicted and recomputed. The second level, `MEMORY_AND_DISK`, spills overflow partitions to local disk instead of evicting them — this is safer for large datasets. The third level, `DISK_ONLY`, stores everything on disk to save RAM for other operations. The fourth level adds replication for fault tolerance. The rule of thumb is simple: start with `.cache()`, and if you see out-of-memory errors, switch to `MEMORY_AND_DISK`.

When to Cache — Decision Flowchart



Cache These

- DataFrames used by **multiple actions**
- **Training data** before `model.fit()`
- DataFrames after **expensive joins**

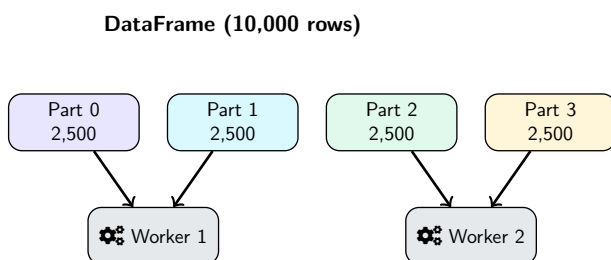
Don't Forget!

Call `df.unpersist()` when done. Cached data stays in memory until the app ends or you free it.

Instructor Notes

To decide whether to cache, follow this two-question flowchart. First: will you use this DataFrame more than once? If no, do not cache — caching a DataFrame you use only once actually wastes memory and adds overhead. If yes, second question: does it fit in RAM? If yes, use `.cache()`. If no, use `.persist(MEMORY_AND_DISK)`. The three most common scenarios where caching helps are: DataFrames used by multiple actions, training data before machine learning, and DataFrames that result from expensive joins. It is important to note that you must call `df.unpersist()` when you are done, because cached data stays in executor memory until the application ends or you explicitly free it.

What Are Partitions?



Analogy

4 TAs, 100 exams. Give each TA 25 = fast. Give 1 TA all 100 = slow. Partitions split data across cores the same way.

```
# Check partitions  
df.rdd.getNumPartitions()
```

Key Rule

1 partition = 1 task. Each task runs on one core of one executor.

Instructor Notes

Partitions are how Spark splits your data for parallel processing. Imagine you have ten thousand rows and four partitions — each partition gets two thousand five hundred rows. Each partition becomes one task, and tasks run in parallel across your executor cores. The analogy is simple: if you have four teaching assistants and one hundred exams, you give each TA twenty-five exams and they all grade in parallel. If you give one TA all one hundred exams, the other three sit idle. Partitions work the same way — they distribute data across cores. You can check how many partitions a DataFrame has using `df.rdd.getNumPartitions()`. The key rule to remember is: one partition equals one task, and each task runs on one core.

The Goldilocks Problem — Finding the Right Partition Count

Too Few (1 partition)

1 core works
3 cores sit idle

SLOW

Just Right (8)

All 4 cores busy
2 rounds of work

FAST

Too Many (200)

Scheduling overhead
Mostly empty tasks

SLOW

The Formula

Sweet spot = $2-4 \times$ total executor cores. Our cluster: 4 cores → **8–16 partitions** is ideal.

How Partitions Are Decided

Default comes from the data source: **HDFS** → one partition per 128 MB block; **CSV** → depends on file splits. Override with `repartition(n)` or `coalesce(n)`.

Instructor Notes

Finding the right partition count is a Goldilocks problem. On the one hand, too few partitions means most cores sit idle — if you have one partition and four cores, three cores do nothing. On the other hand, too many partitions creates scheduling overhead — launching and collecting two hundred tasks when each task processes only fifty rows is wasteful. The sweet spot is two to four times the total number of executor cores. For our cluster with four cores, that means eight to sixteen partitions. It is important to note that the default partition count comes from the data source: HDFS creates one partition per 128 megabyte block, and CSV depends on file splits. You can always override the default with `repartition()` or `coalesce()`, which we will see on the next slide.

`repartition()` vs `coalesce()`

<code>repartition(n)</code>	<code>coalesce(n)</code>
Full shuffle (expensive)	No shuffle (narrow dependency)
Can increase or decrease	Can only decrease
Even distribution guaranteed	Some partitions may be larger
Use for: heavy computation	Use for: writing output files

```
# Increase partitions (full shuffle)
df = df.repartition(8)

# Decrease partitions (no shuffle -- just merge)
df_out = df.coalesce(2)

# Partition by column (co-locate related rows)
df = df.repartition(4, "District")
```

Sweet Spot

Set partitions to **2–4 × total executor cores**. Our cluster: 4 cores → 8 partitions is a good default.

Instructor Notes

Spark gives you two methods to change the partition count, and it is critical to know when to use each one. `repartition(n)` performs a full shuffle — it redistributes all data evenly across the new partitions. This is expensive but guarantees balanced partitions. Use it when you need to increase the partition count or when you need perfectly even distribution for heavy computation. `coalesce(n)` merges existing partitions without a shuffle, which makes it much cheaper — but it can only decrease the count, and the resulting partitions may be unevenly sized. Use it when writing output files and you want fewer files. You can also pass a column name to `repartition()` to co-locate rows with the same key, which is useful before joins.

Shuffle Partitions — The #1 Setting to Change

After any **shuffle** (GROUP BY, JOIN, etc.), Spark creates `spark.sql.shuffle.partitions` output partitions — default is **200**.

```
# BAD for small clusters: 200 tasks on 4 cores!  
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

200 partitions on 4 cores

$200 \div 4 = 50$ rounds

$10,000 \text{ rows} \div 200 = 50$ rows each

Scheduling overhead $\gg$ actual work

8 partitions on 4 cores

$8 \div 4 = 2$ rounds

$10,000 \text{ rows} \div 8 = 1,250$ rows each

Each core stays busy — efficient!

Always Set This!

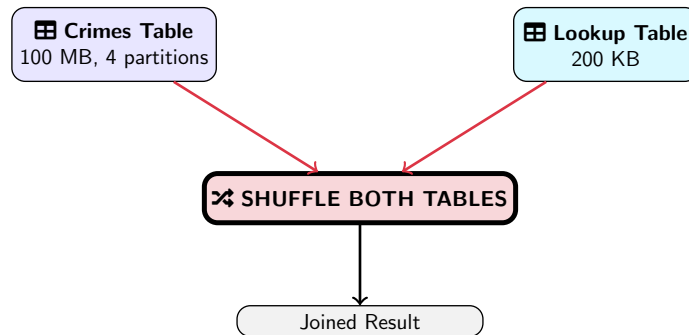
Add `--conf spark.sql.shuffle.partitions=8` to every `spark-submit` command on our cluster. This single setting can **halve** your job execution time.

Instructor Notes

This is arguably the single most impactful setting you can change on a small cluster. After any shuffle — `groupBy`, `join`, `orderBy` — Spark creates a number of output partitions defined by `spark.sql.shuffle.partitions`. The default is two hundred, which is designed for large production clusters with hundreds of cores. On our four-core cluster, two hundred partitions means fifty rounds of scheduling, and each partition has only fifty rows — the scheduling overhead far exceeds the actual computation. If you change this to eight, you get two rounds, each partition has one thousand two hundred fifty rows, and every core stays busy. This single configuration change can halve your job execution time. Always add `--conf spark.sql.shuffle.partitions=8` to your `spark-submit` commands on our cluster.

Outline

The Shuffle Join Problem



Shuffles 100 MB + 200 KB across network = WASTEFUL

Problem

Default join shuffles **both** tables. Shuffling 100 MB of crimes data just to join with a 200 KB lookup table is extremely wasteful.

Navigation icons: back, forward, search, etc.

Instructor Notes

Imagine that you need to join your crimes table, which is one hundred megabytes, with a small lookup table that maps district IDs to district names, which is only two hundred kilobytes. By default, Spark uses a shuffle join, which means it reshuffles both tables across the network so that matching keys end up on the same partition. This means moving one hundred megabytes of crime data plus two hundred kilobytes of lookup data across the network. As a consequence, you are moving one hundred megabytes of data just to match it with a tiny table. This is extremely wasteful. The question we need to answer is: can we avoid shuffling the large table entirely? The answer is yes, using broadcast joins, which we will see on the next slide.

Broadcast Join — The Formula Sheet Analogy

✗ Shuffle Join

Like making **every student** line up at **one shared** formula sheet taped to the wall.

Everyone moves, everyone waits, chaos ensues. (= network shuffle)

✓ Broadcast Join

Like **photocopying** the 1-page formula sheet and giving a copy to **every student**.

Students stay seated, work in parallel. (= local join)

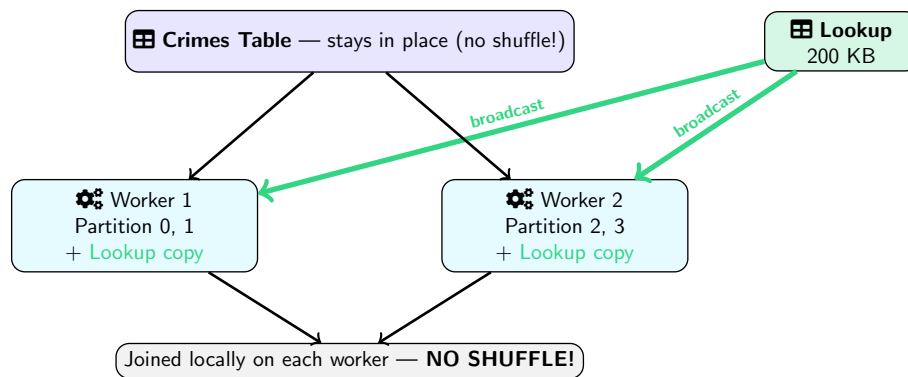
'A' How It Works

- The small Lookup table (200 KB) is **copied entirely** to every executor
- The large Crimes table **stays in place** — partitions don't move
- Each executor joins locally — **no shuffle!**
- Network savings: **99.6% less data** transferred

Instructor Notes

Let us understand this through an analogy. A shuffle join is like having one formula sheet taped to the wall — every student has to get up, walk to the wall, look up their formula, and walk back. Everyone moves, everyone waits, chaos ensues. A broadcast join is like photocopying the formula sheet and giving a copy to every student. Students stay seated and work in parallel. In Spark terms, the small lookup table is copied entirely to every executor. The large crimes table stays in place — its partitions do not move at all. Each executor joins its local partitions with its local copy of the lookup table. The network savings are dramatic: instead of shuffling one hundred megabytes, you only send two hundred kilobytes to each executor — that is 99.6 percent less data transferred.

Broadcast Join — The Solution (Diagram)



Result

Small table (<10 MB) is copied to every executor. Join happens **locally** — zero shuffle of the large table!

Instructor Notes

This diagram shows the mechanics clearly. The crimes table stays distributed across its original partitions — Worker 1 keeps partitions zero and one, Worker 2 keeps partitions two and three. The small lookup table is broadcast — meaning copied in its entirety — to every executor. Now each worker has everything it needs to perform the join locally. No data from the crimes table ever moves across the network. The result is that the join completes with zero shuffle of the large table. The rule is simple: if one table is small, typically less than ten megabytes, broadcast it. Spark actually does this automatically for tables under ten megabytes, but it is good practice to use the `broadcast()` hint explicitly to make your intention clear.

Broadcast Join in Code

```
from pyspark.sql.functions import broadcast

# Small lookup table
district_df = spark.createDataFrame([
    (1, "Central"), (2, "Wentworth"),
    (8, "Chicago Lawn"), (11, "Harrison")
], ["DistrictID", "DistrictName"])

# WITHOUT broadcast (shuffles both tables)
result = df.join(district_df,
    df["District"] == district_df["DistrictID"])

# WITH broadcast (sends lookup to all executors)
result = df.join(broadcast(district_df),
    df["District"] == district_df["DistrictID"])
```

Auto-Broadcast

Spark auto-broadcasts tables < 10 MB

Verify with explain()

Look for BroadcastHashJoin in
result.explain()

Instructor Notes

Let us look at the code. First we create a small lookup DataFrame with district IDs and names. Without the broadcast hint, the join shuffles both tables. With `broadcast(district_df)`, the lookup table is sent to every executor and the join happens locally. It is important to note that Spark automatically broadcasts tables smaller than ten megabytes, controlled by the `spark.sql.autoBroadcastJoinThreshold` setting. However, I recommend always using the explicit `broadcast()` hint for two reasons: first, it documents your intention in the code, and second, it works even if the auto-broadcast threshold has been changed. To verify that Spark is actually using a broadcast join, call `result.explain()` and look for `BroadcastHashJoin` in the physical plan.

Join Strategy Summary

Scenario	Strategy	Cost
Both tables large	Shuffle (Sort-Merge) Join	High (full shuffle)
One table small (<10 MB)	Broadcast Join	Low (no shuffle)
Same partition key	Co-partitioned join	Medium (pre-repartition)
Frequent joins on same key	Bucket join	Low (pre-bucketed)

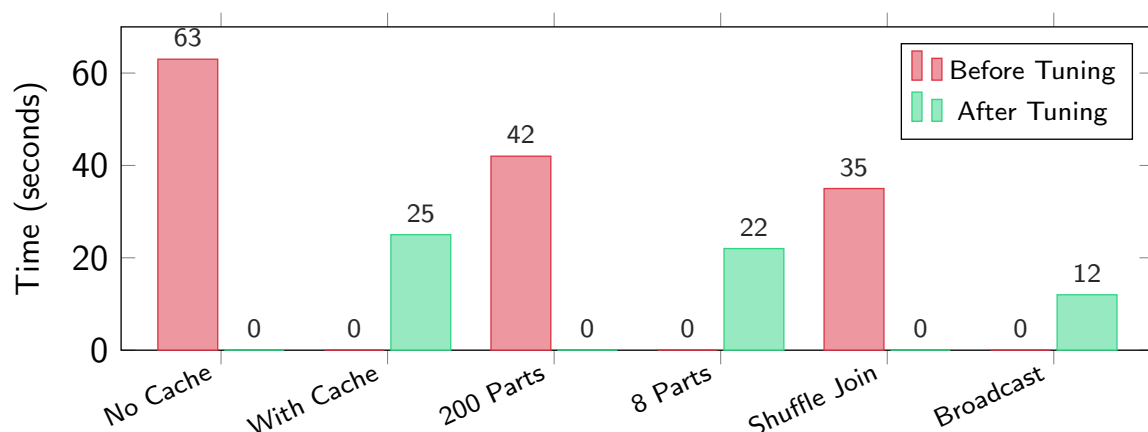
💡 Key Insight

In Big Data courses, most “lookup” joins (district names, payment types, status codes) involve a **small dimension table**. Always use `broadcast()` for these!

Instructor Notes

This table summarizes the four join strategies available in Spark. The first is the shuffle join, also called sort-merge join, which shuffles both tables — this is the default and the most expensive. The second is the broadcast join, which we just learned — it is the cheapest when one table is small. The third is the co-partitioned join, where both tables are already partitioned on the join key. The fourth is the bucket join, where data is pre-bucketed during writing. For this course, the key insight is that most of your joins will involve a small dimension table — district names, crime categories, status codes. These are always small enough to broadcast. As such, always wrap them in `broadcast()` and you will avoid unnecessary shuffles.

Performance Benchmarks — Real Numbers



Cache: $2.5\times$ faster — **Partitions:** $1.9\times$ faster — **Broadcast:** $2.9\times$ faster
Combined: all three levers together can yield $5\text{--}10\times$ speedup on real workloads.

Instructor Notes

These are real benchmark numbers from our cluster running the Chicago Crimes dataset. Caching gave us a 2.5 times speedup by eliminating repeated HDFS reads. Setting shuffle partitions from two hundred to eight gave us a 1.9 times speedup by reducing scheduling overhead. Using broadcast joins instead of shuffle joins gave us a 2.9 times speedup by eliminating network transfer of the large table. Individually, each lever provides a significant improvement. Combined, all three levers together can yield a five to ten times speedup on real workloads. These are not theoretical numbers — you will reproduce them in today's lab and see the exact same patterns in the Spark Web UI.

Key Takeaways

- ❶ **spark-submit**: deploy .py scripts to YARN (client for dev, cluster for production)
- ❷ **Web UI**: Jobs → Stages → Tasks. Watch shuffle bytes, straggler tasks, GC time
- ❸ **Caching**: `.cache()` keeps data in RAM; avoids repeated HDFS reads
- ❹ **Partitions**: set to 2–4× total cores. Use `repartition()` to increase, `coalesce()` to decrease
- ❺ **Broadcast joins**: send small tables to all executors; eliminates shuffle of the large table

The Three Levers of Spark Performance

Lever 1: Caching — Lever 2: Partitioning — Lever 3: Broadcast

Instructor Notes

Let us summarize the five key takeaways from today. First, `spark-submit` is how you deploy scripts to the cluster — use client mode for development and cluster mode for production. Second, the Spark Web UI shows you the job-stage-task hierarchy and helps you spot bottlenecks like shuffle spill, stragglers, and high garbage collection time. Third, caching avoids repeated HDFS reads when you perform multiple actions on the same DataFrame. Fourth, partition tuning ensures all cores stay busy without excessive scheduling overhead. Fifth, broadcast joins eliminate shuffles when one table is small. Remember the three levers: caching, partitioning, and broadcast. Master these three and your Spark jobs will run significantly faster.

Spark Tuning Cheat Sheet — Screenshot This!

>_ `spark-submit` Template:

```
spark-submit --master yarn --deploy-mode client \  
--num-executors 2 --executor-memory 1g --executor-cores 2 \  
--conf spark.sql.shuffle.partitions=8 my_script.py
```

 Caching	 Partitions	'A' Broadcast
Multiple actions on same DF? → <code>df.cache()</code>	Set to 2–4× total cores Our cluster: 8 partitions	Small table < 10 MB? → <code>broadcast(small_df)</code>
OOM? <code>.persist(M&D)</code>	<code>repartition(n) / coalesce(n)</code>	Verify: <code>result.explain()</code>
Done? <code>df.unpersist()</code>	Always set <code>shuffle.partitions!</code>	Never broadcast large tables

Web UI Quick Reference

While running: `http://master:4040` — After finish:
`http://master:8088/proxy/<app-id>/`

Instructor Notes

This is your cheat sheet — take a screenshot of this slide. The `spark-submit` template at the top is the exact command you will use for M3. The table below summarizes the three levers in a decision-oriented format. For caching: if you use a `DataFrame` multiple times, cache it, and unpersist when done. For partitions: set to eight on our cluster and always configure `shuffle.partitions`. For broadcast: wrap small tables in `broadcast()` and verify with `explain()`. The Web UI URLs at the bottom are what you need to monitor your jobs. Keep this slide handy — it contains everything you need to write efficient Spark jobs on our cluster.

Top 5 Student Mistakes — Avoid These!

- ❌ **Using cluster mode during development**
You see no output → use `client` mode for debugging
- ❌ **Expecting `.cache()` to work immediately**
It's lazy! Caching happens on the first **action** after the call
- ❌ **Leaving shuffle partitions at 200**
200 tasks on 4 cores = 50 rounds of overhead. Set to 8!
- ❌ **Forgetting `spark.stop()`**
Your app hogs cluster resources. Always call it at the end.
- ❌ **Broadcasting a large table**
Causes `OutOfMemoryError` on every executor. Only broadcast < 10 MB.

✅ The Fix Pattern

Client mode → cache if reused → set shuffle partitions to 8 → broadcast small tables → `spark.stop()` at the end.

Instructor Notes

These five mistakes come from grading student submissions every semester, so pay attention. The first mistake is using cluster mode during development — you see no output and waste time wondering what happened. The second is expecting `.cache()` to work immediately — it is lazy, just like transformations. The third is leaving shuffle partitions at the default of two hundred — on our four-core cluster, this creates massive scheduling overhead. The fourth is forgetting `spark.stop()` — your application holds onto cluster resources and blocks other students from running their jobs. The fifth is broadcasting a large table, which copies it to every executor and causes out-of-memory errors. The fix pattern at the bottom is your checklist: client mode, cache if reused, set shuffle partitions, broadcast small tables, and always call `spark.stop()`.

Putting It All Together — Optimized Script

```
# optimized_analysis.py -- applies all three tuning levers
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, broadcast

spark = SparkSession.builder \
    .appName("OptimizedCrimeAnalysis") \
    .config("spark.sql.shuffle.partitions", 8) \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)
df = df.repartition(8)      # Lever 2: match core count
df.cache()                 # Lever 1: reuse across actions

df.count()                 # triggers cache
top_crimes = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")).orderBy(col("cnt").desc())
top_crimes.show(10)        # reads from RAM!

# Lever 3: broadcast small lookup table
districts = spark.createDataFrame([(1,"Central"),(2,"Wentworth")],
                                   ["id","name"])
joined = df.join(broadcast(districts), df["District"]==districts["id"])

df.unpersist()             # free memory
spark.stop()               # release cluster resources
```

Instructor Notes

This script brings together everything we learned today. Let us walk through it systematically. First, the `SparkSession` is configured with `shuffle.partitions` set to eight. Then we read the CSV from HDFS and immediately repartition to eight partitions — that is lever two. Next, we call `.cache()` and trigger it with `df.count()` — that is lever one. The subsequent `groupBy` and `show` read from RAM. Then we create a small lookup table and join it using `broadcast()` — that is lever three. Finally, we call `unpersist()` to free memory and `spark.stop()` to release cluster resources. This is the pattern you should follow for your M3 submission. Every line has a purpose, and every tuning decision is intentional.

What's Next

Week 8B: Spark MLlib — Machine Learning at Scale

- ML Pipeline API: Transformer, Estimator, Pipeline
- Feature engineering: `StringIndexer`, `VectorAssembler`
- Train a Random Forest to predict arrests in Chicago crimes
- Evaluate: AUC-ROC, accuracy, F1 score, confusion matrix

Today's Lab: Spark Performance Tuning

- Experiment 1: With vs without caching (3 actions) — target: $>2\times$ speedup
- Experiment 2: Partition count (1, 4, 8, 16) — find the sweet spot
- Experiment 3: Broadcast vs shuffle join — measure network savings
- Record timings + Web UI screenshots for each experiment

Milestone M3 Reminder

Due end of Week 8: RDD + DataFrame + MLlib analytics on Chicago Crimes.

Instructor Notes

Looking ahead, Week 8B will cover Spark MLlib where you will build machine learning pipelines using the same tuning techniques we learned today. You will train a Random Forest model to predict arrests in the Chicago Crimes dataset. For today's lab, you have three experiments to run. The first compares performance with and without caching — you should see at least a two times speedup. The second varies the partition count from one to sixteen to find the sweet spot. The third compares broadcast joins versus shuffle joins. For each experiment, record the execution time and take a screenshot of the Spark Web UI. These results feed directly into your M3 milestone, which is due at the end of Week 8.